Line-by-Line Code Explanation: Automated File Organizer

This document walks through the script from top to bottom. Each section shows a piece of code, followed by a plain-English explanation of what every line does and why it is there.

Big picture: the program works in three steps.

1. **Scan** – find the files that should be organized.
2. **Plan** – decide where each file will go (and rename it if the name is already taken).
3. **Execute** – actually move the files (skipped in dry-run mode).

---

1. Shebang and Docstring

```
#!/usr/bin/env python3
"""
Automated File Organizer  -  Systems
Utility & Automation
Organizes files into Images / Documents /
```

```
Archives / Media / Code (+ Others)
with safe duplicate handling, dry-run
preview, logging and error handling.
...
"""
```

- `#!/usr/bin/env python3` – the "shebang" line. On Linux/macOS it tells the system to run this file with Python 3. On Windows it is simply ignored.
- The triple-quoted text is the **module docstring**. It describes the program and shows example commands. It does not affect how the program runs.

---

2. Imports

```
import argparse
import os
import shutil
import sys
from collections import Counter
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
```

- `argparse` – reads command-line arguments such as `--dry-run` and `-r`.

- `os` – used for `os.walk()`, which walks through folders and subfolders.
- `shutil` – used for `shutil.move()`, which moves files.
- `sys` – used for `sys.exit()` and checking `sys.stdin.isatty()`.
- `Counter` – a dictionary that counts things (used for statistics).
- `dataclass` – a decorator that creates simple data-holding classes with very little code.
- `datetime` – gets the current date and time for logs and timestamps.
- `Path` – a modern, cross-platform way to work with file paths.

---

3. Category Rules

```
CATEGORIES = {
    "Images": {".jpg", ".jpeg", ...},
    "Documents": {".pdf", ".doc", ...},
    "Archives": {".zip", ".rar", ...},
    "Media": {".mp3", ".wav", ...},
    "Code": {".py", ".js", ...},
}
```

- `CATEGORIES` is a dictionary. Each **key** is a folder name, and each **value** is a **set** of file extensions

that belong in that folder.

- Sets are used (curly braces with no key/value pairs) because checking "is this extension in the set?" is very fast and duplicates are impossible.

```
OTHER = "Others"
ALL_FOLDERS = list(CATEGORIES) + [OTHER]
EXT_MAP = {ext: cat for cat, exts in
CATEGORIES.items() for ext in exts}
```

- `OTHER = "Others"` – the folder name for files whose extension is not recognized.
- `ALL_FOLDERS` – a list of all folder names: the five categories plus `Others`. `list(CATEGORIES)` gives just the keys.
- `EXT_MAP` – a **dictionary comprehension** that flips the data around. It loops over every category and every extension in it, producing pairs like `".jpg": "Images"`. This lets the program look up a category directly from an extension.

```
def categorize(path: Path) -> str:
    return EXT_MAP.get(path.suffix.lower(),
OTHER)
```

- `categorize` takes a file path and returns its category name.

- `path.suffix` is the extension (for example `.JPG`). `.lower()` converts it to lowercase so `.JPG` and `.jpg` are treated the same.
- `EXT_MAP.get(key, OTHER)` looks up the extension; if it is not found, it returns `"Others"` instead of crashing.

---

4. Logging

```
class Log:
    """Prints to the console and
(optionally) appends to a log file."""
```

A small helper class that sends messages to the screen and, if requested, to a file as well.

```
    def __init__(self, path=None):
        self.path = path
        self.fh = open(path, "a",
encoding="utf-8") if path else None
```

- `__init__` runs when a `Log` object is created.
- `self.path` stores the log file path (or `None`).
- `self.fh` ("file handle") opens the file in **append mode** (`"a"`) with UTF-8 encoding, so older logs are not erased. If no path was given, it stays `None` and nothing is written to a file.

```
    def write(self, msg=""):
        print(msg)
        if self.fh:
            self.fh.write(msg + "\n")
            self.fh.flush()
```

- Always prints the message to the console.
- If a log file is open, it also writes the message plus a newline.
- `flush()` forces the text to be saved to disk immediately, so the log is not lost if the program crashes.

```
    def close(self):
        if self.fh:
            self.fh.close()
```

- Closes the log file properly (only if one was opened).

---

5. The `Op` Data Class

```
@dataclass
class Op:
    src: Path
    dest: Path
```

```
    category: str
    renamed: bool = False
```

- `@dataclass` automatically generates the constructor and other boilerplate.
- An `Op` ("operation") describes one planned move:
  - `src` – where the file is now.
  - `dest` – where it will go.
  - `category` – which folder category it belongs to.
  - `renamed` – `True` if the name had to change to avoid a clash (default `False`).

---

6. Helper: `rel`

```
def rel(p: Path, base: Path) -> str:
    try:
        return str(p.relative_to(base))
    except ValueError:
        return str(p)
```

- Produces a short, readable path for display, relative to a base folder (for example `sub\report.pdf` instead of the full path).
- If `p` is not inside `base`, `relative_to` raises `ValueError`, and the function falls back to the

full path.

---

7. Avoiding Overwrites: `unique_path`

```
def unique_path(dest: Path, strategy: str,
reserved: set):
    """Return (path, renamed). Never
returns a path that exists or is
reserved."""
```

Finds a file name that is safe to use. It returns two values: the path, and whether it was renamed.

```
    def taken(p):
        return p.exists() or str(p).lower()
in reserved
```

- A nested helper. A name is "taken" if the file already exists on disk **or** if it has already been planned for another file (`reserved`).
- `.lower()` makes the check case-insensitive, matching how Windows treats file names.

```
    if not taken(dest):
        return dest, False
```

- If the name is free, use it unchanged. `False` means "not renamed".

```
    stem, suffix = dest.stem, dest.suffix
```

- Splits `photo.jpg` into `stem = "photo"` and `suffix = ".jpg"`.

```
    if strategy == "timestamp":
        stamp =
datetime.now().strftime("%Y%m%d_%H%M%S")
        cand = dest.with_name(f"
{stem}_{stamp}{suffix}")
        if not taken(cand):
            return cand, True
        stem = f"{stem}_{stamp}"
# same second -> add counter too
```

- With the `timestamp` strategy, a candidate like `photo_20261015_103000.jpg` is built.
- `with_name` creates a new path in the same folder with a different file name.
- If that name is free, it is returned (renamed = `True`).
- If it is *also* taken (two clashes in the same second), the stem is updated to include the timestamp and the code continues to the counter loop below.

```
    n = 1
    while True:
        cand = dest.with_name(f"{stem}_{n}
```

```
{suffix}")
        if not taken(cand):
            return cand, True
        n += 1
```

- The counter approach: try `photo_1.jpg`, then `photo_2.jpg`, and so on until a free name is found.
- This loop is guaranteed to end because eventually an unused number is reached.

---

8. Scanning: `scan`

```
def scan(source: Path, target: Path,
recursive: bool, include_hidden: bool,
         exclude: set):
    """Collect files to organize. Returns
(files, skipped, errors)."""
    files, skipped, errors = [], [], []
    skip_dirs = {(target / c).resolve() for
c in ALL_FOLDERS}
```

- Takes the source folder, target folder, and options.
- Creates three empty lists: files to organize, items skipped (with a reason), and errors.
- `skip_dirs` is a set containing the full paths of the category folders (`Images`, `Documents`, ...).

The program must not re-organize files that are already inside these folders.

The inner function `consider`

```
    def consider(p: Path):
        if p.is_symlink():
            skipped.append((p, "symbolic
link"))
        elif p.name.startswith(".") and not
include_hidden:
            skipped.append((p, "hidden
file"))
        elif p.resolve() in exclude:
            skipped.append((p, "log/script
file"))
        elif not p.is_file():
            skipped.append((p, "not a
regular file"))
        else:
            files.append(p)
```

Decides what to do with one item, checking conditions in order:

1. Symbolic links are skipped (moving them could cause surprises).
2. Hidden files (names starting with `.`) are skipped unless `--include-hidden` was given.
3. The script itself and the log file are skipped (in `exclude`) so the program never moves itself.

4. Anything that is not a regular file is skipped.
5. Otherwise, the file is added to the list to be organized.

The inner function `on_error`

```
    def on_error(e):
        errors.append((Path(e.filename or
"?"), f"cannot read folder: {e.strerror}"))
```

- Called by `os.walk` when a folder cannot be read (for example, permission denied). It records the problem instead of crashing.

Recursive scanning

```
    if recursive:
        for root, dirs, names in
os.walk(source, onerror=on_error):
            root_p = Path(root)
            keep = []
            for d in sorted(dirs):
                dp = root_p / d
                if dp.is_symlink():
                    skipped.append((dp,
"symlinked directory"))
                elif dp.resolve() in
skip_dirs:
                    continue
# already-organized folders
                elif d.startswith(".") and
```

```
not include_hidden:
                    skipped.append((dp,
"hidden directory"))
                else:
                    keep.append(d)
            dirs[:] = keep
            for n in sorted(names):
                consider(root_p / n)
```

- `os.walk` visits every folder, giving the current folder (`root`), its subfolders (`dirs`), and its files (`names`).
- For each subfolder, the code decides whether to enter it: symlinked folders are skipped, category folders are silently ignored, hidden folders are skipped unless allowed, and the rest go into `keep`.
- `dirs[:] = keep` modifies the list **in place**, which tells `os.walk` to only descend into the folders in `keep`. (Assigning without `[:]` would not work.)
- Each file in the current folder is passed to `consider`. `sorted` keeps the order predictable.

Non-recursive scanning

```
    else:
        try:
            entries =
```

```
sorted(source.iterdir(), key=lambda p:
p.name.lower())
        except OSError as e:
            errors.append((source, f"cannot
read folder: {e.strerror}"))
            return files, skipped, errors
        for p in entries:
            if p.is_dir() and not
p.is_symlink():
                if p.resolve() not in
skip_dirs:
                    skipped.append((p,
"subdirectory (use --recursive)"))
            else:
                consider(p)
    return files, skipped, errors
```

- Lists only the top level of the source folder, sorted alphabetically ignoring case.
- If the folder cannot be read, the error is recorded and the function returns early.
- Subfolders are reported as skipped with a hint to use `--recursive` (except category folders, which are quietly ignored). Everything else goes through `consider`.
- Finally the three lists are returned.

---

9. Planning: `build_plan`

```
def build_plan(files, target: Path,
strategy: str):
    reserved, ops = set(), []
    for src in files:
        cat = categorize(src)
        dest, renamed = unique_path(target
/ cat / src.name, strategy, reserved)
        reserved.add(str(dest).lower())
        ops.append(Op(src, dest, cat,
renamed))
    return ops
```

- `reserved` remembers every destination already planned, so two files with the same name get different destinations.
- For each file: find its category, build the ideal destination (`target/Category/filename`), make it unique, record it in `reserved` , and store the plan as an `Op` .
- Because the plan is created **before** moving anything, a dry-run predicts exactly what the real run will do.

---

10. Displaying the Plan: `show_plan`

```
def show_plan(ops, skipped, errors, source,
target, log, label):
    log.write(f"\n{label} planned
```

```
operations ({len(ops)} file(s)):")
    log.write("-" * 70)
```

- Prints a header with the number of planned moves, followed by a line of 70 dashes.

```
    for op in ops:
        note = "  (renamed: name already
exists)" if op.renamed else ""
        log.write(f"  MOVE  {rel(op.src,
source)}  ->  {rel(op.dest, target)}
{note}")
    if not ops:
        log.write("  (nothing to move)")
```

- Prints one `MOVE source -> destination` line per operation, with a note if the file will be renamed.
- If there are no operations, it says so.

```
    if skipped:
        log.write(f"\nSkipped
({len(skipped)}):")
        for p, why in skipped:
            log.write(f"  SKIP  {rel(p,
source)}  [{why}]")
    for p, why in errors:
        log.write(f"  ERROR {p}  [{why}]")
```

- Lists skipped items with the reason, and any errors.

```
    counts = Counter(op.category for op in
ops)
    if counts:
        log.write("\nBy category: " +
                  ", ".join(f"{c}:
{counts[c]}" for c in ALL_FOLDERS if
counts[c]))
```

- Counts how many files go to each category and prints a summary such as `Images: 4, Documents: 3`. Categories with zero files are left out.

---

11. Moving Files: `execute`

```
def execute(ops, strategy, source, target,
log):
    stats = Counter()
    for op in ops:
        try:
```

- `stats` counts results (`moved`, `renamed`, `failed`). Each move is wrapped in `try` so one failure does not stop the rest.

```

op.dest.parent.mkdir(parents=True,
exist_ok=True)
            dest = op.dest
```

```
            if dest.exists():
# changed since planning
                dest, _ = unique_path(dest,
strategy, set())
            shutil.move(str(op.src),
str(dest))
```

- Creates the category folder if it does not exist (`parents=True` creates missing parents, `exist_ok=True` avoids an error if it already exists).
- Re-checks that the destination is still free, because something may have changed since planning; if not, it picks a new name.
- Moves the file.

```
            stats["moved"] += 1
            if dest != op.dest or
op.renamed:
                stats["renamed"] += 1
            log.write(f"  MOVED
{rel(op.src, source)}  ->  {rel(dest,
target)}"
                      + ("  (renamed)" if
dest != op.dest or op.renamed else ""))
```

- Updates the counters and logs the successful move, marking it `(renamed)` if the name changed.

```
        except PermissionError as e:
            stats["failed"] += 1
            log.write(f"  FAILED
{rel(op.src, source)}  [permission denied /
"
                      f"file locked:
{e.strerror or e}]")
        except FileNotFoundError:
            stats["failed"] += 1
            log.write(f"  FAILED
{rel(op.src, source)}  [file no longer
exists]")
        except OSError as e:
            stats["failed"] += 1
            log.write(f"  FAILED
{rel(op.src, source)}  [{e.strerror or
e}]")
    return stats
```

- Three specific failure cases are handled: permission denied or file locked, file disappeared, and any other operating-system error. Each is counted and logged.
- `PermissionError` and `FileNotFoundError` are subclasses of `OSError`, so they must come **before** the general `OSError` handler.
- The function returns the statistics.

---

12. Test Folder Generator:

`make_test_dir`

```
def make_test_dir(path: Path):
    path.mkdir(parents=True, exist_ok=True)
    (path / "Images").mkdir(exist_ok=True)
    (path / "Images" /
"photo.jpg").write_text("existing photo
(already organized)")
    (path / "sub").mkdir(exist_ok=True)
    (path / "sub" /
"nested").mkdir(exist_ok=True)
```

- Creates the test folder, plus an existing `Images` folder containing `photo.jpg`. This lets you test duplicate handling: the new `photo.jpg` will clash with it.
- Also creates `sub` and `sub/nested` folders to test recursive mode.

```
    files = {
        "photo.jpg": "new photo",
"logo.png": "png", ...
        "README": "no extension",
"data.unknownext": "mystery",
        ".hidden_config": "hidden file",
        "sub/report.pdf": "report v2 (same
name as root one)",
        "sub/nested/photo.jpg": "another
photo.jpg",
```

```
        ...
        "UPPER.JPG": "uppercase extension",
    }
```

- A dictionary of file names and their contents, intentionally covering many cases: every category, a file with no extension, an unknown extension, a hidden file, same-named files in different folders, and an uppercase extension.

```
    for name, content in files.items():
        (path / name).write_text(content)
```

- Creates each file with its text content.

```
    print(f"Test directory created at:
{path.resolve()}")
    print(f"Try:  python file_organizer.py
\"{path}\" --dry-run")
    print(f"Then: python file_organizer.py
\"{path}\" --recursive")
```

- Prints where the folder was made and suggests commands to try next.

---

13. Command-Line Interface

`prompt_yes_no`

```
def prompt_yes_no(question, default=False):
    hint = "Y/n" if default else "y/N"
    ans = input(f"{question} [{hint}]:
").strip().lower()
    return default if not ans else
ans.startswith("y")
```

- Asks a yes/no question. The capital letter in the hint (`Y/n` or `y/N`) shows the default.
- `input(...)` reads what the user types; `.strip().lower()` removes spaces and lowercases it.
- If the user just presses Enter, the default is returned. Otherwise it returns `True` when the answer starts with `y`.

`interactive`

```
def interactive(args):
    print("=== Automated File Organizer
(interactive) ===")
    while True:
        s = input("Source folder to
organize: ").strip().strip('"')
        if s and
Path(s).expanduser().is_dir():
            args.source = s
            break
        print("  ! Not a valid folder, try
again.")
```

- Shows the banner you see in the terminal, then keeps asking for the source folder until a valid one is given.
- `.strip('"')` removes quotation marks, which is useful when pasting a path copied from Windows.
- `expanduser()` turns `~` into the home folder. `is_dir()` confirms the folder exists.

```
    t = input("Target folder (Enter = same
as source): ").strip().strip('"')
    args.target = t or None
    d = input("Duplicate names - 'counter'
(file_1) or 'timestamp' "
              "[counter]:
").strip().lower()
    args.duplicates = d if d in ("counter",
"timestamp") else "counter"
    args.recursive = prompt_yes_no("Include
subfolders?", False)
    args.dry_run = prompt_yes_no("Dry-run
only (preview, no changes)?", True)
    return args
```

- Asks for an optional target folder (empty input becomes `None`, meaning "same as source").
- Asks for the duplicate strategy, falling back to `counter` for anything invalid.
- Asks whether to include subfolders (default No) and whether to do a dry-run (default Yes,

the safe choice).

- Returns the filled-in settings.

`parse_args`

```
def parse_args(argv=None):
    p =
argparse.ArgumentParser(description="...")
    p.add_argument("source", nargs="?",
help="...")
```

- Creates the argument parser. `nargs="?"` makes `source` optional, which is what allows interactive mode when it is missing.

```
    p.add_argument("-t", "--target",
help="destination root (default: source)")
    p.add_argument("-n", "--dry-run",
action="store_true", help="...")
    p.add_argument("-r", "--recursive",
action="store_true", help="...")
    p.add_argument("-d", "--duplicates",
choices=["counter", "timestamp"],
                   default="counter",
help="...")
    p.add_argument("--include-hidden",
action="store_true", help="...")
    p.add_argument("--log", metavar="FILE",
help="...")
    p.add_argument("--no-log",
action="store_true", help="do not write a
log file")
```

```
    p.add_argument("-y", "--yes",
action="store_true", help="...")
    p.add_argument("--make-test-dir",
metavar="PATH", help="...")
    return p.parse_args(argv)
```

- Each line defines one option with a short and/or long name.
- `action="store_true"` makes it an on/off switch (`True` when present).
- `choices=[...]` limits the allowed values and shows an error for anything else.
- `parse_args(argv)` reads the command line (or a supplied list) and returns the results.

---

14. The Main Function

```
def main(argv=None):
    args = parse_args(argv)

    if args.make_test_dir:

make_test_dir(Path(args.make_test_dir).expa
nduser())
        return 0
```

- Reads the arguments. If `--make-test-dir` was given, it creates the demo folder and stops

with exit code `0` (success).

```
    interactive_mode = args.source is None
    if interactive_mode:
        args = interactive(args)
```

- If no source folder was given on the command line, the interactive questions are asked. (This is the mode your screenshot shows.)

```
    source =
Path(args.source).expanduser().resolve()
    if not source.is_dir():
        print(f"Error: source '{source}' is
not a folder.")
        return 2
    if source == Path(source.anchor):
        print("Error: refusing to organize
a filesystem root.")
        return 2
    target =
Path(args.target).expanduser().resolve() if
args.target else source
    if target.exists() and not
target.is_dir():
        print(f"Error: target '{target}' is
not a folder.")
        return 2
```

- Converts the source to a full absolute path and validates it.

- Refuses to run on a drive root such as `C:\`, which would be dangerous.
- Sets the target to the given folder, or to the source if none was given, and checks that it is not an existing file.
- Exit code `2` signals invalid input.

```
    log_path = None
    if not args.no_log:
        if args.log:
            log_path =
Path(args.log).expanduser().resolve()
        elif not args.dry_run:
            log_path = Path.cwd() /
f"organizer_log_{datetime.now():%Y%m%d_%H%M
%S}.txt"
    log = Log(log_path)
```

- Decides where to log. With `--no-log`, no log file is written. A custom `--log` path is used if provided. Otherwise, real runs get an automatic timestamped file, while dry-runs create no file at all (so a dry-run leaves the disk untouched).

```
    exclude = {Path(__file__).resolve()}
    if log_path:
        exclude.add(log_path)
```

- Builds the set of files the organizer must never move: the script itself (`__file__`) and the log file.

```
    mode = "DRY-RUN" if args.dry_run else
"EXECUTE"
    log.write(f"File Organizer  |
{datetime.now():%Y-%m-%d %H:%M:%S}  |
mode: {mode}")
    log.write(f"Source : {source}")
    log.write(f"Target : {target}")
    log.write(f"Options: recursive=
{args.recursive}, duplicates=
{args.duplicates}, "
              f"hidden=
{args.include_hidden}")
```

- Prints a header summarizing the mode, folders, and options.

```
    files, skipped, errors = scan(source,
target, args.recursive,

args.include_hidden, exclude)
    ops = build_plan(files, target,
args.duplicates)
    show_plan(ops, skipped, errors, source,
target, log,
              "DRY-RUN" if args.dry_run
else "PLAN:")
```

- Runs the first two steps: scan for files, then plan the moves, then show the plan to the user.

```
    if args.dry_run:
        log.write("\nDry-run complete. No
files were modified.")
        log.close()
        return 0
```

- In dry-run mode, the program stops here. Nothing has been changed.

```
    if not ops:
        log.write("\nNothing to do.")
        log.close()
        return 0
```

- If there is nothing to move, it finishes early.

```
    if not args.yes and sys.stdin.isatty():
        if not prompt_yes_no(f"\nMove
{len(ops)} file(s) now?", False):
            log.write("Cancelled by user.
No files were modified.")
            log.close()
            return 0
```

- Asks for final confirmation (default No) unless `--yes` was passed.

- `sys.stdin.isatty()` is `True` only when a person is typing at a terminal, so automated scripts are not blocked by the prompt.

```
    log.write("\nEXECUTING:")
    log.write("-" * 70)
    stats = execute(ops, args.duplicates,
source, target, log)
    log.write("-" * 70)
    log.write(f"Done. Moved:
{stats['moved']}  |  Renamed to avoid
overwrite: "
              f"{stats['renamed']}  |
Failed: {stats['failed']}  |  "
              f"Skipped: {len(skipped)}  |
Read errors: {len(errors)}")
    if log_path:
        log.write(f"Log saved to:
{log_path}")
    log.close()
    return 1 if (stats["failed"] or errors)
else 0
```

- Performs the real moves, prints a summary line, and mentions the log file location.
- Closes the log and returns exit code `1` if anything failed or could not be read, otherwise `0`.

---

15. Program Entry Point

```
if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\nInterrupted. Files already
moved were not lost.")
        sys.exit(130)
```

- `if __name__ == "__main__":` runs the code below only when the file is executed directly, not when it is imported by another script.
- `main()` runs the program, and `sys.exit(...)` passes its return value to the operating system as the exit code.
- If the user presses Ctrl+C, a `KeyboardInterrupt` is caught, a friendly message is shown, and the program exits with code `130` (the standard code for Ctrl+C). Files that were already moved stay where they are.

Exit Code Summary

Code	Meaning
`0`	Success (including dry-run, nothing to do, or user cancelled)
`1`	Finished, but some moves failed or some folders could not be read
`2`	Invalid source or target folder
`130`	Interrupted with Ctrl+C

Key Concepts Used

- **Dictionary comprehension** – building `EXT_MAP` in one line.
- **Dataclass** – the `Op` class for storing move plans.
- **Nested functions** – `taken`, `consider`, and `on_error` keep helpers close to where they are used.
- **`pathlib.Path`** – clean, cross-platform file path handling.
- **Plan-then-execute design** – makes dry-runs accurate and real runs safer.

- **Exception handling** – `try/except` so one bad file never stops the whole run.